Sprint 4

for

ResistorClassification

Professor Woodward

Prepared by Nolan Tuttle, Mathew Hobson

Software Development Life Cycle II

3/22/26

Table of Contents

Project Overview *3*

Video Link 3

Objective 3

System Architecture 4

Scope 5

Stakeholders 6

Implementation Plan *6*

Evaluation and Performance Analysis 6

Future Testing and Improvement 7

Project Phases and Major Steps 8

Phase 1 – Infrastructure Setup and Environment Configuration 8

Phase 2 – Dataset Preparation and Validation 8

Phase 3 – Model Development and Training Pipeline 9

Phase 4 – Inference and Business Logic Integration 9

Phase 5 – Evaluation, Optimization, and Validation 9

Infrastructure Setup Summary 10

Model Development Strategy 10

Monitoring and Logging 11

Project Timeline Overview 11

Test & Validation Plan *12*

Testing Objectives 12

Test Data 14

Success Criteria (Proof of Concept Acceptance Criteria) 14

Appendix *15*

REQUIREMENTS 15

RISK ASSESSMENT AND MANAGEMENT 21

Project Overview

Video Link

https://youtu.be/5t297C3FIVw

Objective

The purpose of this project is to design and develop a mobile application that identifies the resistance value of a through-hole resistor by analyzing its color bands from a photograph. The application will use computer vision and machine learning techniques to detect the resistor within an image, classify the visible color bands, and calculate the corresponding resistance value using the standard resistor color code system.

The primary goal of the project is to build a functional prototype that demonstrates accurate band classification under typical real-world conditions, including varied lighting and potentially cluttered backgrounds. The application will emphasize reliability, usability, and practical performance on a mobile device while showcasing the integration of image processing and machine learning within a real-world engineering tool.

System Architecture

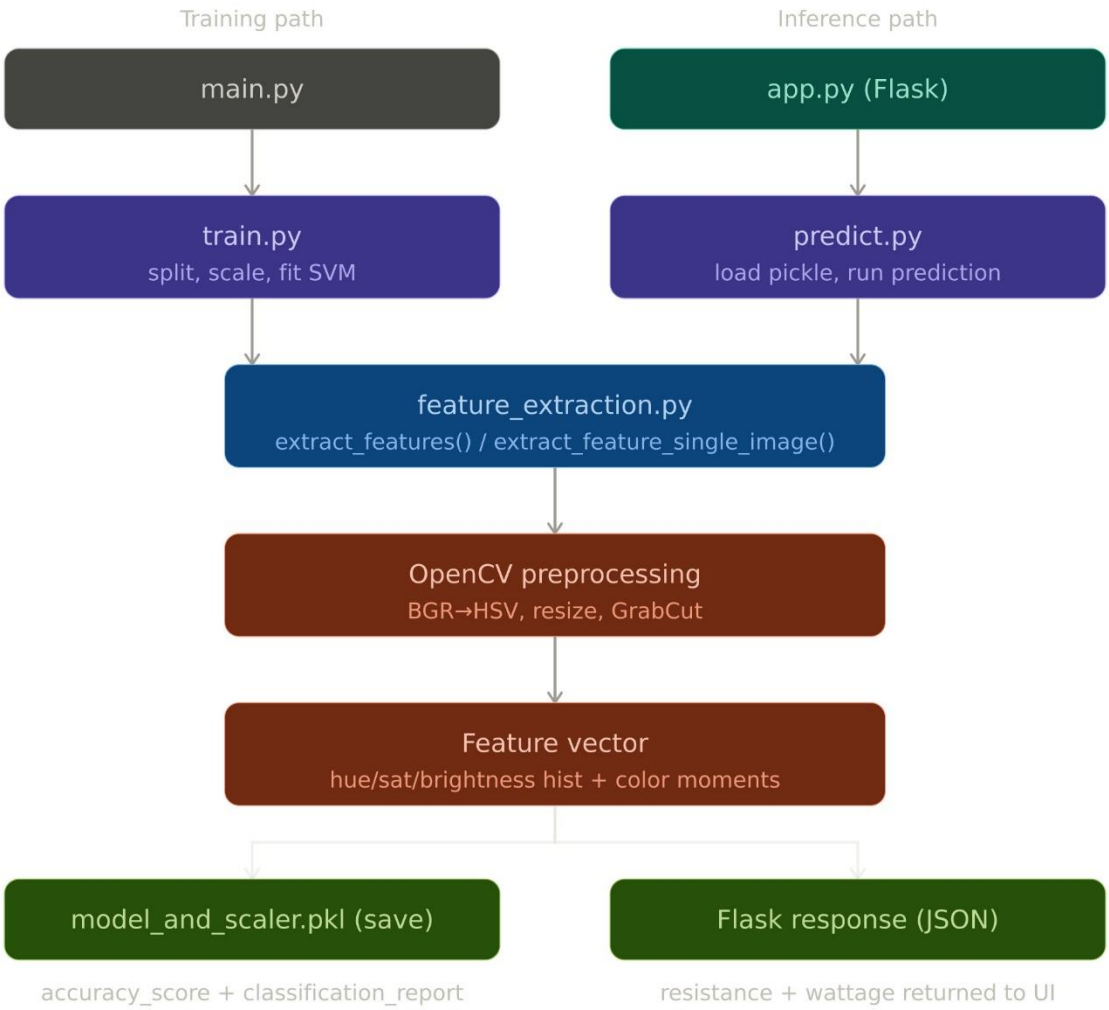

Figure 1: ResistorClassification System Architecture

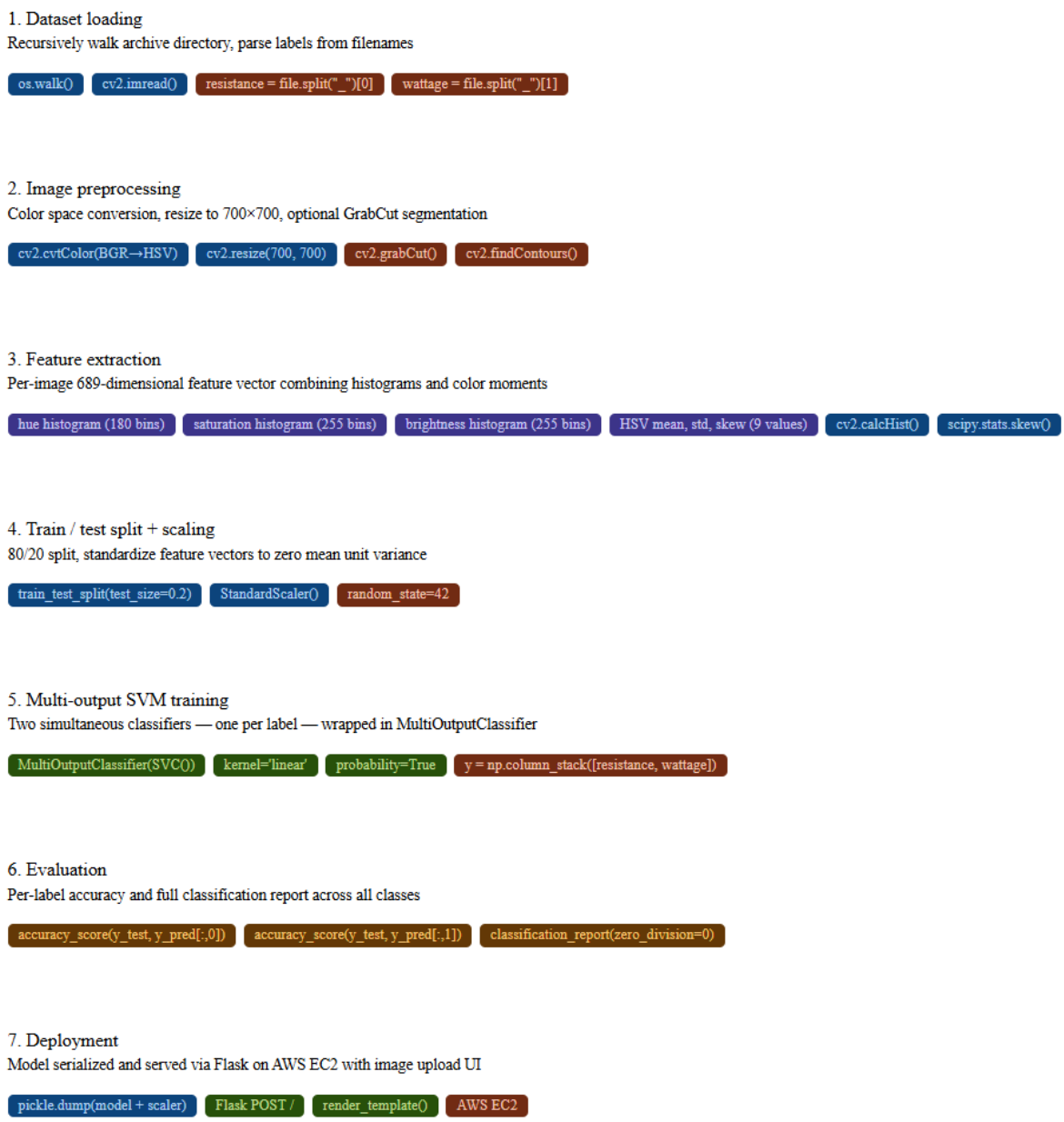

Figure 2: Dataset Processing/Model Training/Deployment Pipeline

Scope

The scope of this prototype includes capturing or uploading an image through a mobile device, detecting the resistor within the image, segmenting the color bands, classifying each band using a trained machine learning model, and computing the final resistance value. The system will support standard 4-band and 5-band resistors and will be optimized for common lighting conditions and standard cylindrical through-hole resistors.

The prototype will focus on accurate color classification and correct resistance computation rather than advanced interface features. It will not include support for surface-mount resistors (SMD), heavily damaged components, extreme lighting conditions, or industrial-scale scanning environments. Additionally, large-scale deployment, cloud-based model training, and enterprise-level optimization are outside the boundaries of this initial implementation.

Stakeholders

The primary stakeholders are the two project developers, both software engineers, who are responsible for system architecture, mobile application development, model training, testing, and documentation. Responsibilities will include dataset collection and labeling, implementation of image processing algorithms, integration of the trained model into the mobile environment, and validation of classification accuracy.

Secondary stakeholders include potential end users such as electronics students, hobbyists, and engineers who may use the application as a practical tool for resistor identification. In an academic context, the course instructor serves as an additional stakeholder by defining project requirements and evaluating the final prototype.

Together, the development team is responsible for delivering a working mobile prototype that demonstrates technical feasibility, accuracy, and practical usability within the defined project scope.

Implementation Plan

This section outlines the phased implementation strategy for the **ResistorClassification** system, incorporating the technical approach defined in the Sprint 1 documentation. The implementation plan is structured around infrastructure setup, model development, integration, monitoring, and validation milestones.

Evaluation and Performance Analysis

Model training data before contour detection and cropping

```
          820R      0.75      0.88      0.81        17
           8K2      0.64      0.60      0.62        15

      accuracy                           0.67       539
     macro avg      0.65      0.64      0.63       539
  weighted avg      0.68      0.67      0.67       539

                precision    recall  f1-score   support

          1-2W      0.73      0.88      0.80        49
          1-4W      0.87      0.88      0.88       331
          1-8W      0.56      0.67      0.61        15
            1W      0.68      0.70      0.69        70
            2W      0.75      0.55      0.64        74

      accuracy                           0.81       539
     macro avg      0.72      0.74      0.72       539
  weighted avg      0.81      0.81      0.80       539

Predicted Resistance: 6R8
Predicted Wattage: 1-4W
```

After contour detection and cropping

```
    accuracy                           0.54       539
   macro avg       0.52      0.53      0.51       539
weighted avg       0.54      0.54      0.53       539

              precision    recall  f1-score   support

        1-2W       0.66      0.67      0.67        49
        1-4W       0.81      0.85      0.83       331
        1-8W       0.50      0.67      0.57        15
          1W       0.62      0.54      0.58        70
          2W       0.67      0.55      0.61        74

    accuracy                           0.75       539
   macro avg       0.65      0.66      0.65       539
weighted avg       0.74      0.75      0.74       539
```

Future Testing and Improvement

After implementing the automatic cropping feature, a decrease in model accuracy was observed, likely due to inconsistent or imperfect cropping that may remove or distort important color band information.

Future improvements will focus on refining the cropping algorithm by adjusting contour thresholds and ensuring the full resistor is consistently captured. Additional testing will compare performance with and without cropping, and the dataset may be updated to include similarly cropped images to better align training and inference conditions. Alternative approaches, such as more precise detection methods, may also be explored to improve reliability.

Project Phases and Major Steps

The project will be implemented in five primary phases:

Phase 1 – Infrastructure Setup and Environment Configuration

This phase establishes the AWS and development environment required to support model training and inference.

Steps include:

1. Provisioning an Amazon EC2 t2.micro instance under AWS free-tier eligibility.
2. Installing Ubuntu 24.04 LTS and configuring SSH access via security groups.
3. Installing Python 3.10+ and creating an isolated virtual environment.
4. Installing required dependencies (PyTorch, OpenCV, NumPy, pandas, scikit-learn).
5. Configuring Elastic Block Storage (EBS) volumes for dataset, model artifacts, and logs.
6. Verifying CPU-only training capability within resource limits (≤ 2 vCPU, ≤ 1 GB RAM).

Milestone 1: Cloud instance operational with all dependencies installed and verified.

Phase 2 – Dataset Preparation and Validation

This phase focuses on dataset ingestion and preprocessing consistency.

Steps include:

1. Importing the 3000+ labeled resistor images into the EC2 filesystem.
2. Verifying directory-based labeling structure (folders organized by resistance value).
3. Sanitizing image metadata and validating file integrity.
4. Implementing dataset loader with reproducible 70/15/15 train-validation-test splits using fixed random seeds.
5. Testing preprocessing pipeline (resize, normalization, RGB conversion, cropping).

Milestone 2: Dataset successfully loaded, validated, and split with preprocessing pipeline verified.

Phase 3 – Model Development and Training Pipeline

This phase implements the CNN-based classification model described in Sprint 1.

Steps include:

1. Selecting a CNN architecture (e.g., ResNet-18 or EfficientNet-B0) suitable for CPU-based training.
2. Configuring training hyperparameters:
    - Learning rate: 0.001
    - Batch size: 16–32
    - Maximum epochs: 50
    - Early stopping: patience of 5 epochs
3. Implementing categorical cross-entropy loss for multi-class classification.
4. Integrating validation step after each epoch to track generalization performance.
5. Saving best-performing model checkpoints based on validation accuracy.
6. Logging training metrics (accuracy, loss, precision, recall, F1 score).

Milestone 3: Model achieves ≥95% validation accuracy and meets F1 ≥0.93 performance threshold.

Phase 4 – Inference and Business Logic Integration

This phase integrates the trained model into the inference pipeline.

Steps include:

1. Implementing model loader at system startup.
2. Connecting preprocessing pipeline to inference engine.
3. Implementing band-to-numeric mapping algorithm using standard resistor color code rules.
4. Computing final resistance value (Ω, kΩ, MΩ) including tolerance calculation.
5. Generating structured console output and JSON/CSV formatted outputs.

Milestone 4: End-to-end inference pipeline successfully processes new images and outputs resistance values with confidence scores.

Phase 5 – Evaluation, Optimization, and Validation

This phase ensures system reliability and compliance with non-functional requirements.

Steps include:

1. Running evaluation on test dataset and generating full metrics report.
2. Computing:
    - Overall accuracy (target ≥95%)
    - Exact match accuracy
    - Macro precision/recall
    - F1 score ≥0.93
    - Top-3 accuracy ≥98%
3. Measuring inference latency (≤3 seconds per image).
4. Validating memory usage within EC2 limits.
5. Generating reliability diagrams for confidence calibration.

Milestone 5: All functional and non-functional requirements validated and documented.

Infrastructure Setup Summary

The system infrastructure consists of:

- AWS EC2 t2.micro instance (CPU-only)
- Ubuntu 24.04 LTS
- Python virtual environment
- PyTorch-based model training
- OpenCV image preprocessing
- EBS persistent storage for datasets and models
- requirements.txt for reproducible deployment

All training and inference operations will execute locally on the EC2 instance without GPU acceleration.

Model Development Strategy

The model development process follows a supervised learning approach:

1. Directory-based label extraction from dataset structure.
2. Standardized preprocessing pipeline shared between training and inference.
3. CNN-based multi-class classification model.
4. Early stopping to prevent overfitting.
5. Checkpoint-based model preservation.

6. Post-training evaluation using classification and regression-style metrics.

Transfer learning may be used to accelerate convergence and improve generalization under CPU constraints.

Monitoring and Logging

Monitoring and logging mechanisms ensure traceability and reliability.

Training Monitoring:

- Epoch-level logging of training and validation accuracy
- Loss curves stored for analysis
- Checkpoint saving based on validation improvement

Inference Logging:

- Timestamped logs of predictions
- Confidence scores per prediction
- Error logging for invalid inputs

System Monitoring:

- CPU and memory usage observation during training
- Verification of model load success at startup
- Persistent log storage on EBS

Structured logs will be stored locally and retained for post-run analysis.

Project Timeline Overview

PHASE	DESCRIPTION	DELIVERABLE
PHASE 1	AWS & environment setup	Operational EC2 instance
PHASE 2	Dataset ingestion & preprocessing	Validated dataset pipeline
PHASE 3	Model training & validation	Trained model meeting accuracy targets
PHASE 4	Inference integration	Working classification system
PHASE 5	Testing & optimization	Final evaluation report

Test & Validation Plan

Testing Objectives

The objective of testing is to validate that the **ResistorClassification** prototype functions correctly, meets defined performance thresholds, and operates reliably within the AWS free-tier CPU environment. Testing will focus on validating both functional correctness and machine learning performance.

The following tests will be conducted:

1. Functional Testing

- Verify that the system accepts valid JPEG and PNG image inputs.
- Confirm that invalid or corrupted files are rejected without crashing.
- Validate that preprocessing (resize, normalization, cropping) executes consistently.
- Ensure the trained model loads successfully at startup.
- Confirm that the system outputs:
    - Predicted band sequence
    - Classification confidence
    - Computed resistance value

2. Model Validation Testing

- Evaluate classification performance using validation and test datasets.
- Measure per-class and per-band accuracy.
- Confirm correct band-to-resistance conversion logic.
- Verify tolerance-adjusted resistance accuracy.

3. Performance Testing

- Measure inference latency (must complete ≤ 3 seconds per image on EC2 CPU instance).
- Monitor CPU and memory usage to ensure free-tier compliance.
- Validate that sequential inference requests can be processed without restart.

4. Robustness Testing

- Test images with moderate background clutter.
- Test images with varied lighting conditions.
- Validate system behavior with low-confidence predictions.
- Confirm consistent outputs for identical inputs.

5. Deployment Testing

- Validate full system operation on EC2 t2.micro instance.
- Confirm dependency installation and environment setup via virtual environment.
- Verify persistent storage of logs, datasets, and trained model files.

Test Data

Validation will use the following datasets:

1. Primary Labeled Dataset

- 3,000+ labeled resistor images.
- Organized in folder-based supervised structure (resistance value as directory name).
- Split into:
    - 70% Training
    - 15% Validation
    - 15% Test

Splits will use a fixed random seed to ensure reproducibility.

2. Augmented Validation Samples

- Images with brightness variation.
- Slight rotation and perspective changes.
- Background clutter simulations.

These will be used to evaluate robustness.

3. Real-World Test Set

- Newly captured images not used during training.
- Taken in semi-controlled but realistic conditions.
- Used to evaluate generalization performance.

All EXIF metadata will be sanitized before inclusion in the dataset, in accordance with the data privacy requirements.

Success Criteria (Proof of Concept Acceptance Criteria)

The prototype will be considered successful if it meets the following measurable indicators:

Accuracy Metrics

- ≥ 95% exact resistance classification accuracy on the test set.
- ≥ 98% Top-3 accuracy.
- ≥ 0.93 F1-score (macro averaged).
- ≥ 95% per-band accuracy.
- ≥ 80% of predictions exceed 95% confidence threshold.

Performance Metrics

- ≤ 3 seconds inference time per image on CPU EC2 instance.
- Training completes within 2 hours on free-tier hardware.
- No memory overflow or instance termination during training or inference.

Reliability Metrics

- 100% successful handling of invalid inputs without crash.
- Identical predictions for identical inputs using same model version.
- Persistent logging of inference results and training metrics.

Practical Usability Metrics

- ≥ 95% tolerance-adjusted accuracy (within standard resistor tolerance ranges).
- MAE and RMSE on log-scale remain within acceptable deviation thresholds defined during evaluation.

If all above thresholds are met in both controlled validation and real-world testing, the Proof of Concept will be considered validated and ready for final project delivery.

Appendix

REQUIREMENTS

1. Functional Requirements

1.1. Data Input Requirements

1. FR-DI-1 Image Submission

i. The system shall accept resistor images through command-line input.

ii. The system shall support JPEG and PNG image formats.

2. FR-DI-2 Input Validation

i. The system shall verify file type and file integrity before processing.

ii. The system shall reject unsupported or corrupted inputs.

3. FR-DI-3 Image Preprocessing

i. The system shall preprocess images prior to classification using OpenCV.

ii. Preprocessing shall include:

1. resizing to model input dimensions
2. normalization
3. cropping or isolating the resistor region

4. FR-DI-4 Dataset Handling (Training Mode)

i. The system shall load labeled training images from a dataset directory.

ii. The system shall allow dataset updates without modifying application logic.

1.2. ML Model Requirements

1. FR-ML-1 Model Training

i. The system shall train a machine learning model using labeled resistor images.

ii. The system shall support configuration of training parameters.

iii. The system shall output a model artifact after training.

2. FR-ML-2 Model Loading

i. The system shall load a trained model into memory during startup.

ii. The system shall allow model replacement without modifying preprocessing code.

3. FR-ML-3 Prediction / Inference
    i. The system shall classify resistor band patterns from processed images.
    ii. The system shall return:
        1. predicted band sequence
        2. classification confidence
4. FR-ML-4 Resistance Computation
    i. The system shall convert predicted band colors into resistance values.
    ii. The system shall apply standard resistor color code rules.

1.3. Cloud Service Requirements

1. FR-CL-1 Cloud Execution
    i. The system shall execute training and inference on a cloud-hosted virtual machine using Amazon EC2.
    ii. The system shall operate within CPU-only free-tier resource constraints.
2. FR-CL-2 Environment Setup
    i. The system shall run within a Python virtual environment.
    ii. The system shall install required dependencies during deployment.
3. FR-CL-3 Storage Management
    i. The local filesystem shall store:
        1. Datasets
        2. Trained model files
        3. Log files
        4. Maintain persistent instance storage.
4. FR-CL-4 Runtime Operation
    i. The system shall accept inference requests while running on the cloud instance.

ii. The system shall log processing activity and system errors.

2. Non-Functional Requirements

2.1. Performance Requirements

1. NFR-P-1 Inference Latency

   i. The system shall complete image preprocessing and inference within ≤ 3 seconds per image on a CPU-based cloud instance.

2. NFR-P-2 Training Completion

   i. The system shall complete model training within the computational limits of a free-tier virtual machine (≤ 2 vCPU, ≤ 1 GB RAM).

   ii. Training time shall not exceed 2 hours maximum for the full dataset.

3. NFR-P-3 Resource Utilization

   i. The system shall not exceed available memory during preprocessing or inference.

   ii. CPU utilization shall remain within instance limits without causing system termination.

4. NFR-P-4 Concurrent Requests (Baseline)

   i. The system shall support sequential inference requests without requiring restart.

4.2 Reliability Requirements

1. NFR-R-1 Fault Tolerant Input

   i. The system shall handle invalid image files without crashing.

   ii. The system shall log preprocessing and classification errors.

2. NFR-R-2 Model Stability

   i. The system shall verify successful model load at startup.

3. NFR-R-3 Logging

i. The system shall log:

   1. Inference results

2. Errors
3. Training metrics

ii. Logs shall persist for analysis after model is run

4. NFR-R-4 Reliable Output

i. Given identical inputs and the same trained model, the system shall produce consistent predictions.

3. Data Requirements

3.1. Data Sources and Format

3.1.1. The ResistorClassification project will utilize publicly available resistor image datasets that permit public use for research and educational purposes. The image dataset will be organized into a labeled directory structure where images are stored in folders named according to their resistance values. This organizational approach facilitates supervised learning by allowing the system to extract labels directly from the folder hierarchy.

3.1.2. Image files will follow a consistent naming convention that includes the resistance value and an image sequence number (e.g., 100ohm_001.jpg, 100ohm_002.jpg). This naming scheme enables easy identification and traceability of individual images within the dataset while maintaining clear associations with their corresponding resistance values.

3.2. Data Privacy Requirements

3.2.1. To protect privacy and respect intellectual property rights, the image dataset will exclude any images containing personally identifiable information or proprietary content. Specifically, images will not contain faces, license plates, proprietary circuit designs, company markings, or address location information. This restriction ensures the dataset can be shared and used for research purposes without privacy or legal concerns.

3.2.2. Additionally, EXIF metadata embedded in image files will be sanitized to remove sensitive location and device information. GPS coordinates and

camera serial numbers will be stripped from all images before they are added to the dataset. However, certain non-sensitive metadata such as timestamp, resolution, and camera model will be preserved as this information may be useful for understanding dataset characteristics and image quality variations.

3.3. Data Retention Policy Requirements

3.3.1. The dataset will be retained indefinitely during the active development phase of the project to support ongoing model training and experimentation. However, images that are identified as poor quality or unsuitable for training will be flagged and retained for only 90 days before deletion. This allows time for review and potential correction while preventing the accumulation of low-quality data that could degrade model performance.

4. Deployment Requirements

4.1. Storage and Cloud Computing

4.1.1. The ResistorClassification system will be deployed on Amazon Web Services using an EC2 t2.micro instance to take advantage of AWS free-tier eligibility. The instance will run Ubuntu Server 24.04 LTS, providing a stable Linux environment for the Python-based machine learning application. Security groups will be configured to allow SSH access on port 22 for administrative tasks and system management.

4.1.2. For persistent storage, the system will utilize Elastic Block Storage (EBS) volumes attached to the EC2 instance. Storage allocation will include approximately 20 GB for the operating system and software dependencies, 5 GB for the resistor image dataset, and 2 GB for trained model artifacts and system logs.

4.1.3. The software environment will be built on Python 3.10 or higher, with all dependencies managed through a Python virtual environment to ensure isolation from system packages. Core dependencies include a deep learning framework (PyTorch), OpenCV for image processing, NumPy and pandas for data manipulation, and scikit-learn for evaluation metrics. All package versions will be documented in a requirements.txt file.

RISK ASSESSMENT AND MANAGEMENT

Technical Risks

Risk 1: Insufficient Model Accuracy

- Probability: Medium
- Impact: High
- Description: Model fails to achieve 90% accuracy target, producing unreliable results
- Mitigation Strategies:
    - Collect diverse, high-quality training dataset with multiple lighting conditions
    - Implement robust data augmentation techniques
    - Experiment with multiple model architectures (MobileNet, ResNet, EfficientNet)
    - Allocate additional time for hyperparameter tuning
- Contingency Plan: If accuracy remains below 85% by March 15, pivot to simpler classification problem (fewer resistor types) or implement ensemble methods combining multiple models

Risk 2: Limited Machine Learning Experience

- Probability: High
- Impact: Medium
- Description: Team's limited ML expertise leads to implementation delays or suboptimal solutions
- Mitigation Strategies:
    - Complete online ML courses (Fast.ai, Coursera TensorFlow specialization) during first two weeks
    - Leverage transfer learning to reduce complexity
    - Schedule weekly consultations with instructor or ML-experienced peers
    - Use well-documented frameworks and follow established tutorials
- Contingency Plan: Utilize pre-built ML APIs (Google Vision AI) as fallback if custom model development proves too challenging

Risk 3: Data Collection Challenges

- Probability: Medium
- Impact: Medium
- Description: Difficulty obtaining sufficient diverse resistor images for training
- Mitigation Strategies:
    - Purchase assorted resistor kits (estimated $30-50) for controlled photography
    - Use data augmentation to expand dataset artificially
    - Explore existing open-source resistor image datasets
- Contingency Plan: Generate synthetic resistor images using computer graphics if physical collection is insufficient

Risk 4: Deployment and Integration Issues

- Probability: Medium
- Impact: Medium
- Description: Challenges deploying trained model to web/mobile environment
- Mitigation Strategies:
    - Test deployment early (by March 20) rather than waiting until final phase
    - Use platform-specific optimization tools (TensorFlow Lite, TensorFlow.js)
    - Follow cloud provider documentation closely
    - Maintain fallback option of simpler REST API deployment
- Contingency Plan: Deploy as desktop application or API-only service if web/mobile proves too complex

Resource Risks

Risk 5: GPU Resource Limitations

- Probability: Low-Medium
- Impact: Medium
- Description: Insufficient GPU access for training, leading to extended training times
- Mitigation Strategies:
    - Leverage Google Colab free tier with GPU acceleration • Use team members' personal GPU resources
    - Optimize model architecture for faster training
    - Train during off-peak hours to maximize resource availability

- Contingency Plan: Use smaller model architectures (MobileNet instead of ResNet) or consider cloud GPU rental ($1-2/hour on GCP/AWS)

Risk 6: Budget Constraints

- Probability: Low
- Impact: Low
- Description: Project costs exceed available budget
- Mitigation Strategies:
    - Maximize use of free tiers (GCP $300 credit, AWS free tier)
    - Prioritize essential features, defer nice-to-have elements
    - Use open-source tools and frameworks exclusively
- Contingency Plan: Seek additional funding through GCU computer science department or scale back deployment to local-only demonstration

Schedule Risks

Risk 7: Timeline Compression

- Probability: Medium
- Impact: High
- Description: Unexpected delays in one phase compress subsequent phases, threatening April 26 deadline
- Mitigation Strategies:
    - Build 1-week buffer into schedule (implicit in 14-week timeline) • Identify critical path items and prioritize accordingly
    - Conduct weekly progress reviews to catch delays early
    - Establish minimum viable product (MVP) definition to ensure core functionality is delivered
- Contingency Plan: Reduce scope by eliminating secondary objectives (accessibility features, 5-band resistors) if necessary to deliver core functionality on time

Risk 8: Team Coordination Challenges

- Probability: Low
- Impact: Medium
- Description: Communication or coordination issues between team members
- Mitigation Strategies:
    - Establish regular meeting schedule (twice weekly)
    - Use collaborative tools (GitHub, Slack/Discord, shared documents)
    - Clearly define roles and responsibilities
    - Maintain transparent progress tracking
- Contingency Plan: Instructor mediation if conflicts arise; restructure task assignments to play to individual strengths

Quality Risks

Risk 9: Poor User Experience

- Probability: Low-Medium
- Impact: Medium
- Description: Application interface is confusing or difficult to use
- Mitigation Strategies:
    - Conduct early user testing with engineering students (Week 10)
    - Follow established UI/UX design principles
    - Implement clear error messages and guidance
    - Create simple, intuitive workflow (capture → result in 2 steps)
- Contingency Plan: Rapid iteration based on user feedback; simplify interface to bare essentials if needed

Risk 10: Real-World Performance Gap

- Probability: Medium
- Impact: Medium
- Description: Model performs well on test data but poorly in real-world conditions
- Mitigation Strategies:
    - Include challenging scenarios in test dataset (poor lighting, angled shots, worn resistors)
    - Conduct extensive field testing with actual users
    - Implement confidence thresholding (reject predictions below 80% confidence)
    - Provide user feedback mechanism to identify problem cases

- Contingency Plan: Collect real-world failure cases and retrain model; provide manual fallback option

Risk Monitoring and Review

- Weekly risk assessment during team meetings
- Maintain risk register with current probability/impact ratings
- Escalate high-impact risks to instructor immediately
- Monthly formal risk review with mitigation strategy updates